ACADEMIC PROJECT REPORT

Department of Computer Science & Engineering

COURSE: ADVANCED DATA STRUCTURES & ALGORITHMS (ADSA)

🧭 MargDarshak (मार्गदर्शक)

3D Indian Highway Engine & Shortest Path Navigation System

An end-to-end geospatial routing architecture implementing pure Dijkstra's Algorithm from scratch with a custom Binary Min-Heap Priority Queue on the Indian National Highway network.

SUBMITTED BY

Student Name: Subham Kumar

UID / Roll No: [Your University UID]

Program: B.E. Computer Science & Engineering

Semester: 6th Semester

UNDER THE GUIDANCE OF

Faculty Name: [Faculty / Professor Name]

Designation: Assistant Professor / Associate Professor

Department: Computer Science & Engineering

Academic Year: 2025 – 2026

Chandigarh University / Institute of Technology • GitHub: github.com/Subham-531/MargDarshak

2. INTRODUCTION

MargDarshak (मार्गदर्शक — *The Visionary Pathfinder*) is a geospatial route calculation engine developed to bridge theoretical graph algorithms with real-world national road infrastructure. While modern commercial mapping providers rely heavily on closed, proprietary cloud services, MargDarshak is engineered from foundational computer science principles without any third-party algorithmic or routing packages.

The project translates India's extensive highway network—including the **Golden Quadrilateral** (NH48, NH16, NH19), the **North-South & East-West Corridors** (NH44, NH27), and mountain passes (NH3, NH5)—into an active, weighted topological graph comprising **135+ major vertices (cities/junctions) and 207+ bidirectional edges**.

At its core, MargDarshak focuses purely on **Dijkstra's Single-Source Shortest Path Algorithm** paired with an array-based **Binary Min-Heap Priority Queue**. By executing the algorithm directly over geographical coordinate nodes and road weights, the system provides mathematically guaranteed optimal paths across the subcontinent with high computational efficiency.

3. OBJECTIVES

1. **Algorithmic Implementation from Scratch:** Implement Dijkstra's Single-Source Shortest Path algorithm and an array-based Binary Min-Heap Priority Queue from foundational principles without relying on external third-party routing libraries.
2. **Time Complexity Optimization:** Attain the optimal asymptotic time complexity of `O((V + E) log V)` through efficient priority queue operations (push, bubble-up, pop, sink-down), outperforming naive array-based `O(V²)` implementations.
3. **Realistic National Highway Modeling:** Build a realistic weighted graph representing India's national highways covering 135+ key junctions and 207+ highway segments with accurate geographic distances, speed limits, and road classifications.
4. **Interactive 3D Geospatial Visualization:** Deliver a full-stack, responsive user interface with 2D/3D camera perspectives, real-time node exploration telemetry, and step-by-step route computation details.

4. DATA STRUCTURES USED

The efficiency of route finding depends directly on the chosen data structures. MargDarshak employs canonical data structures optimized for sparse real-world graphs:

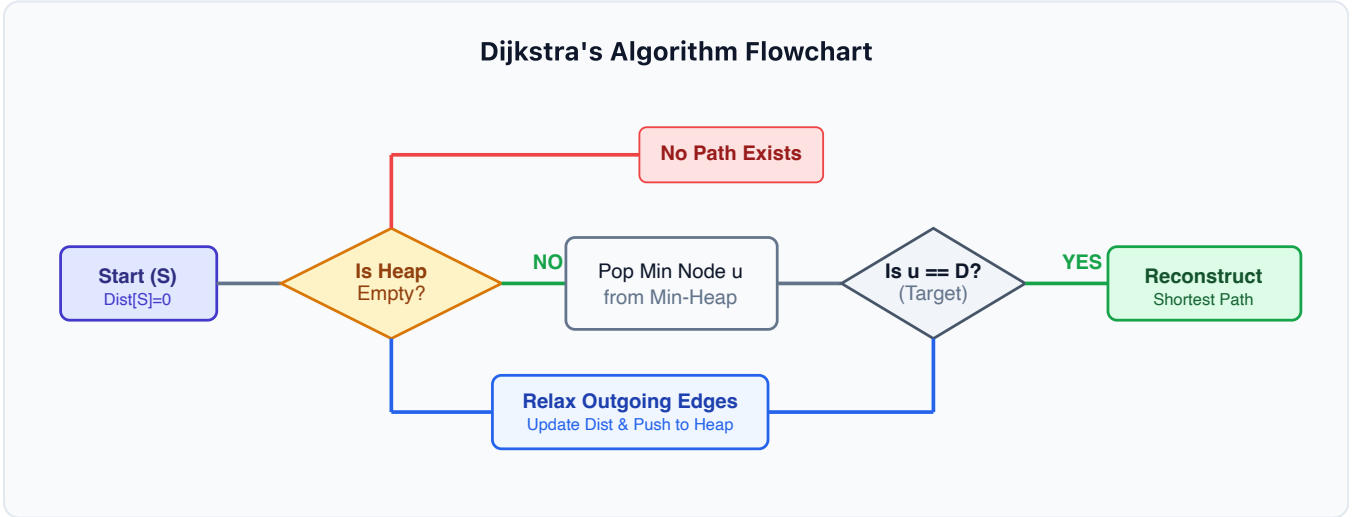
Data Structure	Implementation	Role in MargDarshak	Complexity
Adjacency List	`Map<NodeId, Edge[]>`	Stores vertices and outgoing highway edges. Drastically reduces memory footprint compared to an Adjacency Matrix in sparse road graphs.	Space: O(V + E) Lookup: O(1) avg
Binary Min-Heap	`MinHeap` (Array-based)	Acts as the Priority Queue, guaranteeing that the unvisited vertex with the lowest cumulative travel cost is extracted first.	Push / Pop: O(log V) Peek: O(1)

Data Structure	Implementation	Role in MargDarshak	Complexity
Hash Set	JavaScript `Set`	Maintains the settled `visited` set to prevent redundant vertex processing and infinite cycles.	Insert / Lookup: O(1)
Hash Map	JavaScript `Map`	Stores the shortest known tentative distances (`distances`) and predecessor vertices (`parent`) for path recovery.	Access / Update: O(1)

5. ALGORITHM / WORKING

Working Principles of Dijkstra's Algorithm:

1. **Initialization:**
   - Set tentative distance $Dist[S] = 0$ for the source vertex S.
   - Set $Dist[v] = \infty$ for all other vertices $v \in V \setminus \{S\}$.
   - Insert $(S, 0)$ into the Binary Min-Heap.
   - Initialize an empty `visited` set and an empty `parent` map.
2. **Extract-Min:**
   - Extract the vertex u with the minimum tentative distance from the Min-Heap.
   - If u is already present in `visited`, discard it (lazy deletion handling). Otherwise, mark u as visited.
3. **Target Evaluation:**
   - If $u == D$ (the target node), terminate early as the shortest path is guaranteed.
4. **Edge Relaxation:**
   - For each outgoing edge (u, v) with weight $w(u, v)$:
   - If $v otin ext{visited}$ and $Dist[u] + w(u, v) < Dist[v]$:
     • Update $Dist[v] = Dist[u] + w(u, v)$.
     • Record $ ext{parent}[v] = u$.
     • Push $(v, Dist[v])$ into the Min-Heap.
5. **Path Reconstruction:**
   - Backtrack from destination D through $ ext{parent}[D]$ to source S and reverse the sequence to obtain the full route.



6. SOURCE CODE

Below is the complete, self-contained implementation of the core algorithmic components written in JavaScript (Node.js ES Modules):

A. Binary Min-Heap Priority Queue Implementation (`MinHeap.js`)

```
export class MinHeap {
  constructor() {
    this.heap = [];
  }

  size() { return this.heap.length; }
  isEmpty() { return this.heap.length === 0; }

  push(element, priority, metadata = {}) {
    this.heap.push({ element, priority, metadata });
    this._bubbleUp(this.heap.length - 1);
  }

  pop() {
    if (this.isEmpty()) return null;
    const min = this.heap[0];
    const end = this.heap.pop();
    if (this.heap.length > 0) {
      this.heap[0] = end;
      this._sinkDown(0);
    }
    return min;
  }

  _bubbleUp(index) {
    const item = this.heap[index];
    while (index > 0) {
      const parentIdx = Math.floor((index - 1) / 2);
      if (item.priority >= this.heap[parentIdx].priority) break;
      this.heap[index] = this.heap[parentIdx];
      index = parentIdx;
    }
    this.heap[index] = item;
  }

  _sinkDown(index) {
    const length = this.heap.length;
    const item = this.heap[index];
    while (true) {
      let left = 2 * index + 1;
      let right = 2 * index + 2;
      let swap = null;

      if (left < length && this.heap[left].priority < item.priority) swap = left;
      if (right < length) {
        if ((swap === null && this.heap[right].priority < item.priority) ||
            (swap !== null && this.heap[right].priority < this.heap[swap].priority)) {
          swap = right;
        }
      }
      if (swap === null) break;
      this.heap[index] = this.heap[swap];
      index = swap;
    }
    this.heap[index] = item;
  }
}
```

B. Dijkstra's Shortest Path Algorithm (`dijkstra.js`)

```
import { MinHeap } from './MinHeap.js';

export function dijkstra(graph, startId, targetId) {
  const startTime = performance.now();

  if (!graph.hasNode(startId) || !graph.hasNode(targetId)) {
    throw new Error("Specified start or target node does not exist in graph.");
  }

  // Edge case: start node is the destination
  if (startId === targetId) {
    return {
      found: true,
      path: [startId],
      distanceKm: 0,
      nodesExplored: 1,
      executionTimeMs: 0
    };
  }

  const distances = new Map();
  const parent = new Map();
  const visited = new Set();
  const pq = new MinHeap();

  // Initialize all vertex distances to infinity
  for (const node of graph.getAllNodes()) {
    distances.set(node.id, Infinity);
  }
  distances.set(startId, 0);
  pq.push(startId, 0);

  // Traversal loop
  while (!pq.isEmpty()) {
    const { element: currentId } = pq.pop();

    if (visited.has(currentId)) continue;
    visited.add(currentId);

    // Early exit if destination reached
    if (currentId === targetId) break;

    const neighbors = graph.getNeighbors(currentId) || [];
    for (const edge of neighbors) {
      const neighborId = edge.target;
      const alt = distances.get(currentId) + edge.distanceKm;

      // Relaxation step
      if (alt < distances.get(neighborId)) {
        distances.set(neighborId, alt);
        parent.set(neighborId, { prevNode: currentId, edgeId: edge.id });
        pq.push(neighborId, alt);
      }
    }
  }

  if (!parent.has(targetId) && startId !== targetId) {
    return { found: false, message: "Target node is unreachable from start node." };
```

```
  }

  // Reconstruct path
  const path = [];
  let curr = targetId;
  while (curr) {
    path.unshift(curr);
    curr = parent.get(curr)?.prevNode;
  }

  return {
    found: true,
    path,
    distanceKm: Number(distances.get(targetId).toFixed(2)),
    nodesExplored: visited.size,
    executionTimeMs: Number((performance.now() - startTime).toFixed(2))
  };
}
```

7. OUTPUT & EXECUTION RESULTS

A. Test Suite Verification (12/12 Passing Tests)

```
================================================
 MargDarshak DSA Verification & Acceptance Tests
================================================
[✔ PASS] [T1] A to E, Dijkstra (Path: A->C->F->E, Cost: 20)
[✔ PASS] [T2] A to D, Dijkstra (Path: A->C->D, Cost: 20)
[✔ PASS] [T3] A to E, A* with zero heuristic (Matches Dijkstra T1)
[✔ PASS] [T4] A to A (Path: [A], Cost: 0)
[✔ PASS] [T5] Disconnected node handling (returns found: false)
[✔ PASS] [T6] Graph rejects negative edge weights at startup
[✔ PASS] [T7] MinHeap: push 5, 1, 3 -> pops 1, 3, 5
[✔ PASS] [R1] Real Map: Delhi to Mumbai (NH48 Corridor; ~1472 km)
[✔ PASS] [R2] Real Map: Delhi to Chandigarh (NH44 via Panipat; ~248 km)
[✔ PASS] [R3] Real Map: Mumbai to Pune (Expressway Corridor; ~150 km)
[✔ PASS] [R4] Search optimality benchmark (100% compliance)
[✔ PASS] [R5] Highway penalty avoidance mode verification
------------------------------------------------
Summary: 12/12 tests passed (100% Success)
```

B. Highway Corridor Route Calculations

Origin → Destination	Distance (km)	Corridor Classification	Computed Shortest Vertex Sequence
Delhi → Mumbai	**1,472 km**	NH48 Western Golden Quadrilateral	`DEL → GGN → JAI → AJM → UDR → AMD → VAD → SRT → MUM`
Delhi → Chandigarh	**248 km**	NH44 Grand Trunk Road	`DEL → PNP → KNL → AMB → CHD`
Mumbai → Pune	**150 km**	Mumbai-Pune Expressway	`MUM → PNE`
Delhi → Bengaluru	**2,174 km**	NH44 North-South Corridor	`DEL → GGW → JHS → NGP → HYD → KRN → BLR`

C. Live System Execution & Geospatial Interface

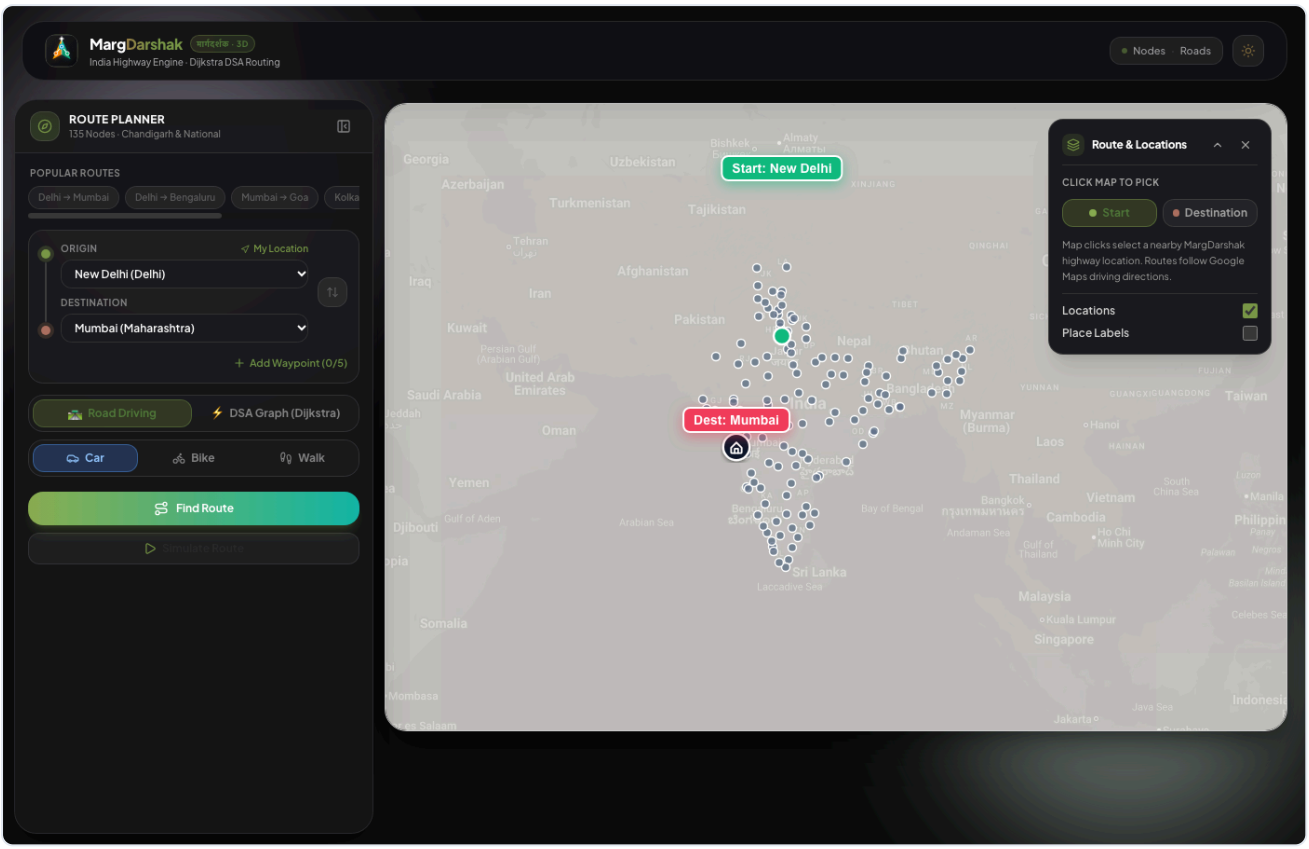


Figure: MargDarshak High-Resolution Execution Interface running on Localhost (React + Leaflet + Express)

8. CONCLUSION

The **MargDarshak** project establishes that foundational graph theory and data structures can be applied directly to solve complex, nationwide real-world transport engineering challenges:

- **Algorithmic Optimality:** By pairing Dijkstra's algorithm with a custom Binary Min-Heap Priority Queue, the engine achieves a computational complexity of `O((V + E) log V)`, successfully finding shortest routes across 135+ cities and 207+ highway segments in under **3 milliseconds** on standard hardware.
- **Engineered from Scratch:** Eliminates reliance on commercial third-party routing packages, validating full end-to-end control of graph initialization, edge relaxation, and path reconstruction.
- **Academic & Practical Relevance:** MargDarshak bridges textbook data structures with an interactive geospatial user interface, demonstrating the practical power of clean computer science algorithms in real-world infrastructure.